









Working with Gilles Dowek on Modeling and Proof Methods in Set Theory using Deduction Modulo Theory

Catherine Dubois[1] David Delahaye[2]

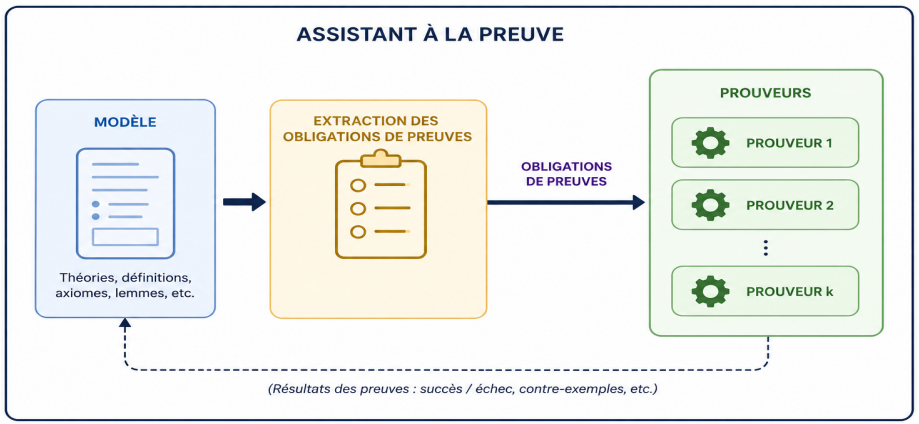
18 July 2026

[1]Samovar, ENSIIE, Évry-Courcouronnes, France

[2]LIRMM, Univ Montpellier, CNRS, Montpellier, France

Proof Assistants Based on Set Theory

- B/Event-B and TLA+ formalisms, based on set theory and the concept of state
- TLA+: specification and verification of concurrent and distributed systems, temporal logic
- TLA+: untyped formalism, special status of functions
  B/Event-B: typed formalism, relations
- Industrial applications (railway systems, cloud services, etc.)
- IDEs: TLAPS, AtelierB and Rodin



Objectives

$\longrightarrow$ More automatic proofs and more confidence in the provers

Developed in 2 ANR French projects

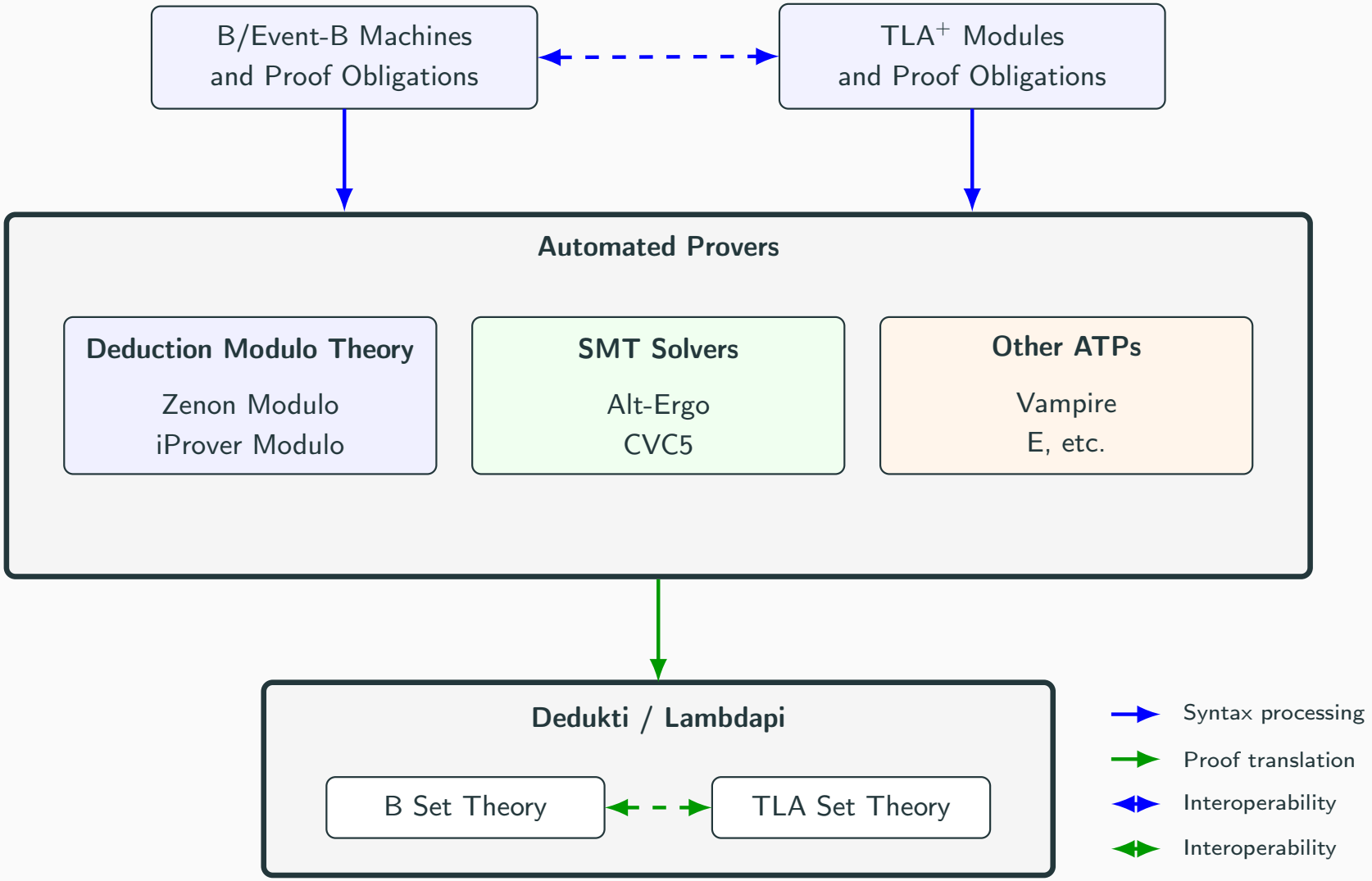
- BWare (2012-2016) (A Proof-Based Mechanized Plate-Forme for the Verification of B Proof Obligations)
- ICSPA (Interoperable and Confident Set-based Proof Assistants) (2022-2026)

BWare: provide a multi-tools verification platform with the requirement for the verification tools to produce proof objects, which are to be checked independently.

ICSPA: check the proofs of the Atelier B/Rodin and TLAPS provers

In both cases, rely on $\lambda\Pi$-calculus modulo and its type-checkers

BWare and ICSPA Approaches



Deduction Modulo Theory

- Theorem Proving Modulo / Deduction Modulo [G. Dowek, Th. Hardin, and C. Kirchner, 2003]

  Separate logical reasoning from computation.
  - Axioms are replaced by computation/rewriting rules.
  - Logical reasoning is performed on formulas modulo the rewriting relation.
- $\lambda\Pi$-calculus modulo [D. Cousineau and G. Dowek, 2007]

  Functional interpretation of proofs.
  - Extension of the LF $\lambda\Pi$-calculus: dependent types + higher-order rewriting rules (confluence, type preservation)
  - Logical Framework.
  - Dedukti [Boespflug, 2011]: type checker for the $\lambda\Pi$-calculus modulo
  - LambdaPi [G. Hondet and F. Blanqui, 2020]): simple set of tactics, standard library, ...
  - Used as an interoperability target for many proof systems (Matita, Coq, HOL, etc.).

Automated Deduction Modulo Theory

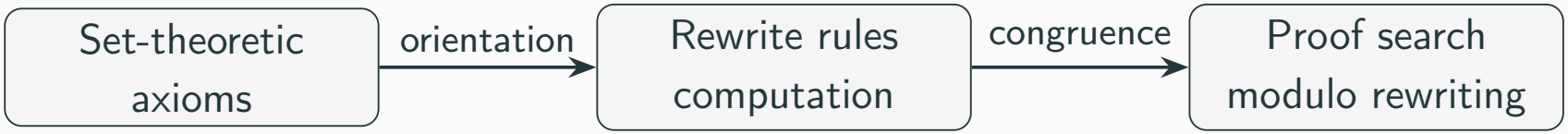
- Deduction Modulo Theory is not only a logical framework:

  it also gives a methodology for proof search.

- Main idea:
  - keep the logical calculus small and generic;
  - move part of the axiomatic theory into computation;
  - reason modulo a rewriting congruence.

- This is particularly useful for set-theoretic formalisms: B, Event-B, TLA+, set operators, relations, functions, etc.

From Axioms to Computation



Logical inference is applied on formulas considered up to computation.

Benefit

Definitions do not have to be unfolded by logical reasoning only: they are handled by computation, which can make proof search more direct.

Several Provers Based on These Ideas

The ideas of Deduction Modulo Theory have led to the development of several automated provers.

- **Super Zenon**: tableaux with superdeduction rules.
  [M. Jacquel, D. Delahaye, C. Dubois, K. Berkani, IJCAR 2012]
- **Zenon Modulo**: tableaux modulo rewriting, with proof output.
  [P. Halmagrand, D. Delahaye, O. Hermant, D. Doligez, F. Gilbert, LPAR 2013]
- **iProver Modulo**: instantiation-based reasoning modulo theories.
  [G. Burel, CADE 2011]
- **ArchSAT**: SAT-based reasoning modulo rewriting.
  [G. Bury, S. Cruanes, D. Delahaye. SMT 2018]
- **Zipperposition**: superposition with theories and rewriting.
  [S. Cruanes, PhD thesis 2015]
- **Goéland**: concurrent proof search with certified proof output.
  [J. Cailler, J. Rosain, D. Delahaye, S. Robillard, H. Bouziane, IJCAR 2022]
- **VIP**: recent prover participating in CASC-J13.
  [I. Nokrani, C. Gérardin, D. Delahaye, CASC-J13 2026]

Why Deduction Modulo Theory for Set Theory?

- Set theory contains many definitions:

$$\subseteq, \quad \cup, \quad \cap, \quad \mapsto, \quad \mathit{dom}, \quad \mathit{ran}, \quad \text{relations}, \quad \text{functions}$$

- Axiomatizing all these notions directly creates many proof-search steps.
- Deduction Modulo Theory turns part of this reasoning into rewriting:

definitions become computation rules.

- This gives automated provers a more natural access to the mathematical content of B/Event-B/TLA+ proof obligations.

Deduction Modulo Theory for the B Set Theory

- The B set theory is typed and is expressed with many type constraints:

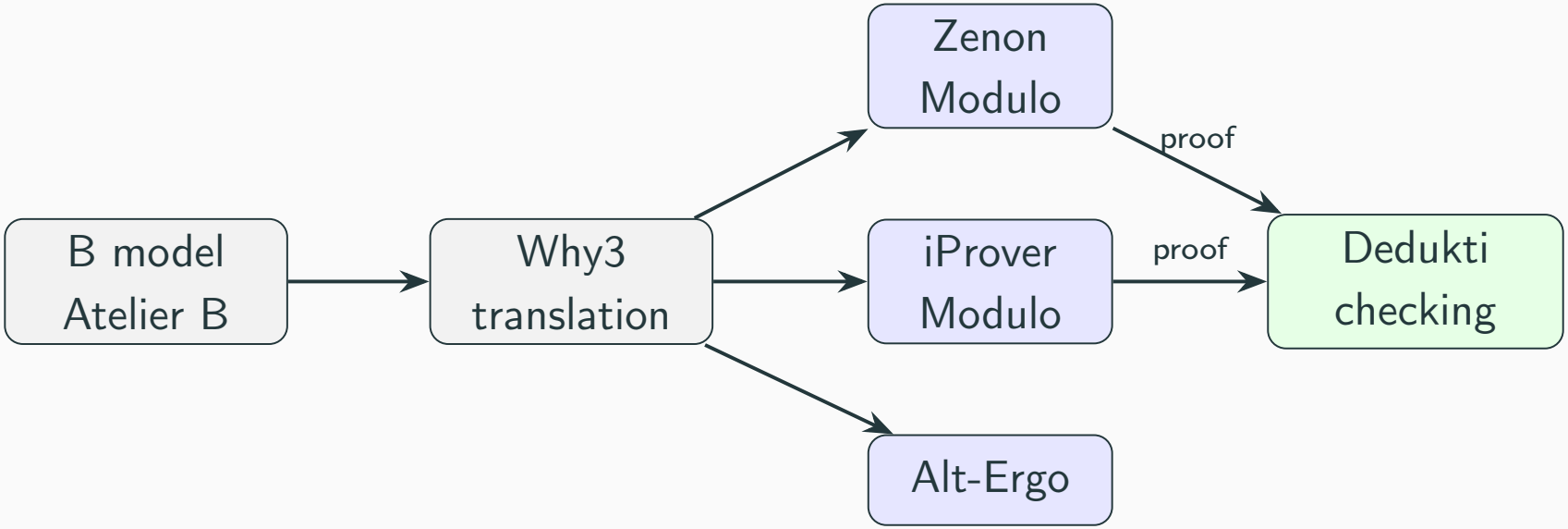
  Translation into Deduction Modulo Theory less straightforward.

- Idea: use a first-order logic with polymorphic types.
  The type constraints are captured by the typing system.
  [P. Halmagrand, G. Bury, D. Delahaye, O. Hermant, D. Doligez, LPAR 2015]

$$\begin{aligned}
&\mathsf{set} : \Pi\alpha : \mathsf{Type}.\mathsf{Type} \\
&\text{- } \in \text{ - } : \Pi\alpha : \mathsf{Type}.\alpha \to \mathsf{set}(\alpha) \to o \\
&\text{- } \cap \text{ - } : \Pi\alpha : \mathsf{Type}.\mathsf{set}(\alpha) \to \mathsf{set}(\alpha) \to \mathsf{set}(\alpha) \\
&x \in_\alpha s \cap_\alpha t \longrightarrow x \in_\alpha s \wedge x \in_\alpha t
\end{aligned}$$

- The ATP must understand Deduction Modulo Theory as well as the polymorphic nature of the logic $\to$ Zenon Modulo.

BWare Verification Workflow



Key points

- Goal: automatically discharge B proof obligations with high confidence.
- The prover is not the final trusted component: proof objects are checked by an independent kernel.

BWare Results

- Large benchmark of B proof obligations (industrial applications):

  about 13,000 problems in set theory.

- Zenon Modulo solved:

  95.4% of the proof obligations.

- For the non-arithmetic part:

  10,340 proofs were produced and checked by Dedukti.

- This demonstrated that automated deduction modulo theory can be used both for automation and for independently checkable proof production (with small certificates).

(Re)Verification of Proofs via Proof Logging

Automatic provers are buggy! We cannot trust them! Especially for developing industrial critical systems!

Verification of proofs is a big concern in the ICSPA project:

- verification of cvc5 proofs for TLA+/TLAPS (A. Coltellacci, S. Merz)
- **verification of PP proofs for B** (C. Dunne, G. Burel, C. Dubois)
- verification of EventB/Rodin interactive proofs (A. Grieu, J.-P. Bodeveix, M. Filali)

Common approach: reconstruct a Lambdapi proof term from a proof trace and let Lambdapi typechecker give the verdict

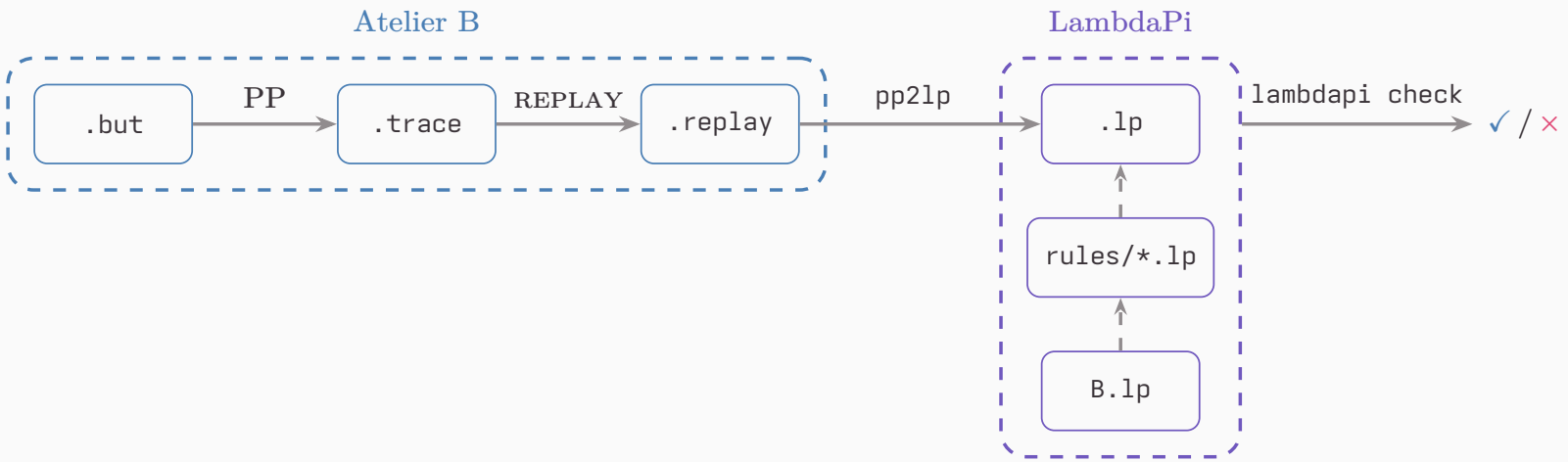
Verification of PP Proofs

PP (Predicate Prover) written around 1995 by Jean-Raymond Abrial, an automatic prover supplied with Atelier B and Rodin

A very effective tool, but . . . a black box, with inaccessible code . . .

However. . . PP generates a compact trace that can be replayed (REPLAY), yielding an expanded trace (each step is annotated with the rule applied and the intermediate goal it was applied to)

⟶ `pp2lp` reconstructs these replays in Lambdapi



Re-construction of a proof term (1)

- (Shallow) Encoding of PP's mathematical language (first-order logic $+ \in$, the maplet $\mapsto$, equality, Booleans and whatever is needed to handle arithmetic rules) built on the Lambdapi standard library
- Definition of the 138 PP inference rules: each rule becomes a Lambdapi symbol whose type states the rule and whose body is a proof script for its soundness

  - Two kinds of judgements

$$\frac{H \vdash Q \qquad H \vdash P}{H \vdash P \wedge Q}\ \text{AND4} \qquad \frac{(H \vdash Q) \rightsquigarrow R_1 \qquad (H \vdash P) \rightsquigarrow R_2}{(H \vdash P \wedge Q) \rightsquigarrow R_1 +\!\!+ R_2}\ \text{AND4_1}$$

  - Arithmetic side-conditions in some rules but no subproof recorded in the replay

Re-construction of a proof term (2)

- Automatic reconstruction of the proof extracted from a *replay*: each replay becomes a Lambdapi symbol whose staement is the goal and whose body is a proof script

PP proof replay

```
[AND2] <not(p and not(p))>
[IMP4] <p => not(not(p))>
[NOT2] <not(not(p))>
[AXM3] <p>
```



LambdaPi reconstruction

```
opaque symbol t02 (p : Prop)
  : π (¬ (p ∧ ¬ p)) ≔
begin
  assume p;
  refine AND2 _;
  refine IMP4 _;
  assume hp;
  refine NOT2 hp
end;
```

Reconstruction of arithmetic side conditions subproofs (Fourier-Motzkin elimination, proof by reflection technique of Coltellacci and Merz)

Evaluation

`og`, `prv`: custom proof-rules from AtelierB developments

`claude`: suite authored by an LLM agent (Claude opus 4.8)

`apero-mini`: industrial proof obligations from CLEARSY's public dataset

Suite	Replays	Verified	Timeouts	Failures
`og`	30	30	0	0
`prv`	70	70	0	0
`claude`	2328	2317	0	11
`apero-mini`	405	390	5	10

A Lasting Scientific Legacy



Deduction Modulo Theory changed the way we think about **automated reasoning**. It inspired new proof methods, **proof certification**, and **independent proof verification**.

Its legacy continues to shape the future of **proof interoperability**.

This is only the beginning.